IOT PROJECT PORTFOLIO

Fingerprint Attendance & Absentee Report via Telegram

Developed by	Sakshi Borade
Class	TY B.Sc. Information Technology – A
Subject	Internet of Things (IoT)
Academic Year	2026–27

This portfolio presents the project as developed by Sakshi Borade, including the system design, Python implementation, fingerprint attendance workflow, LCD output, absentee report and Telegram integration.

1. Project Overview

The Fingerprint Attendance & Absentee Report System is an IoT application built around a Raspberry Pi. A fingerprint sensor identifies registered users. The Python program maps the detected fingerprint template ID to a student record and marks attendance. A 16×2 LCD displays the status. At the end of the session, the program calculates absentees, saves the report as **attendance.txt** and sends it through a Telegram bot.

2. Objectives

• Identify registered students using a fingerprint sensor connected to Raspberry Pi.

• Map fingerprint template IDs to student roll numbers and names.

• Prevent duplicate attendance entries during one session.

• Display student names and attendance status on a 16×2 LCD.

• Generate present and absent lists and save them in a text file.

• Deliver the attendance report through Telegram.

3. Hardware and Software

Component	Purpose
Raspberry Pi	Main processing platform.
Fingerprint sensor	Captures and searches fingerprint templates.
16×2 LCD	Displays names and attendance status.
Python	Main application language.
PyFingerprint	Fingerprint sensor communication.
RPLCD / RPi.GPIO	LCD and GPIO control.
python-telegram-bot	Telegram report delivery.
attendance.txt	Local report storage.

4. System Working

Fingerprint → Identification → Attendance → Report → Absentee List → Telegram

1. Sensor initialization

The Raspberry Pi initializes the fingerprint sensor and verifies its connection.

2. Fingerprint capture

The user places a finger on the sensor.

3. Identification

The captured fingerprint is converted and searched against stored templates.

4. Student mapping

The returned template ID is matched with a student record.

5. Attendance

The student is added to a set, preventing duplicate entries.

6. LCD output

The LCD displays the student name and attendance status.

7. Report generation

The program compares registered IDs with present IDs to calculate absentees.

8. Saving and Telegram

The report is saved locally and sent through the Telegram bot.

5. Main Features

Fingerprint identification: Uses the PyFingerprint library to search stored fingerprint templates.

Duplicate prevention: Uses a Python set to ensure each student is counted once per session.

LCD status: Shows ready, place-finger, present, already-marked, unknown-fingerprint and report-status messages.

Absentee calculation: Calculates absent IDs by subtracting present IDs from all registered IDs.

Report and Telegram: Creates a readable attendance report, writes it to attendance.txt and sends it to a Telegram chat.

6. Testing and Results

The supplied project report documents successful final testing of the end-to-end workflow on Raspberry Pi. The recorded sample result was Total Students: 5, Present: 1 and Absent: 4. The report was saved and delivered to the Telegram bot.

7. Problems and Resolutions

Fingerprint not registered: Checked fingerprint enrollment and the template-ID mapping.

Attendance count issue: Reviewed the student mapping and present/absent calculation.

Library setup: Checked the required Python imports and Raspberry Pi library setup.

Telegram authorization / chat errors: Corrected the bot configuration and chat ID, then retested delivery.

8. Learning Outcome and Conclusion

This project demonstrates integration of IoT hardware with Python software. It combines fingerprint identification, attendance logic, LCD output, report generation, file handling and Telegram messaging in one workflow. The project provided practical experience in hardware/software integration, debugging, testing and presenting results.

Security note: The Telegram token and chat ID are placeholders in the code. Keep real credentials private and never commit them to a public repository.

9. Python Source Code (s.py)

The source code is included below. Student names are represented with sample labels; replace them with your authorized records. Enter real Telegram credentials only in your local copy and keep them out of public uploads.

```
import RPi.GPIO as GPIO
from RPLCD.gpio import CharLCD
from pyfingerprint.pyfingerprint import PyFingerprint
from telegram import Bot
import datetime
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

# Hardware initialization
lcd = CharLCD(
    pin_rs=25,
    pin_rw=None,
    pin_e=24,
    pins_data=[23, 17, 18, 22],
    numbering_mode=GPIO.BCM,
    cols=16,
    rows=2
)

# Telegram configuration
# Add your own private bot token and chat ID locally.
# Never upload real credentials to a public GitHub repository.
TELEGRAM_TOKEN = "YOUR_BOT_TOKEN"
TELEGRAM_CHAT_ID = "YOUR_CHAT_ID"
bot = Bot(token=TELEGRAM_TOKEN)

# Fingerprint sensor initialization
try:
    sensor = PyFingerprint(
        '/dev/ttyUSB0',
        57600,
        0xFFFFFFFF,
        0x00000000
    )

    if not sensor.verifyPassword():
        raise ValueError("Sensor authentication failed!")

    print("&gt;&gt;&gt; Sensor Connected Successfully &gt;&gt;&gt;")

except Exception as e:
    lcd.clear()
    lcd.write_string("Sensor Error!")
    print(f"Error: {e}")
    raise SystemExit(1)

# Student database: fingerprint template ID -&gt; (roll number, name)
# Replace these sample entries with authorized test data.
STUDENT_DATABASE = {
    20: ("12", "Student 1"),
    21: ("02", "Student 2"),
    2: ("11", "Student 3"),
    3: ("06", "Student 4"),
    4: ("47", "Student 5")
}

present_students = set()

# Main attendance loop
lcd.clear()
lcd.write_string("System Ready")
lcd.cursor_pos = (1, 0)
lcd.write_string("Scan Finger...")
print("&gt;&gt; System Ready. Place finger on sensor...")
time.sleep(2)

try:
    while True:
        lcd.clear()
        lcd.write_string("Place Finger...")

        # Wait until a fingerprint is detected
        while not sensor.readImage():
            time.sleep(0.1)

        # Convert and search the fingerprint
        sensor.convertImage(0x01)
        result = sensor.searchTemplate()
        position_number = result[0]

        if position_number &gt;= 0 and position_number in STUDENT_DATABASE:
            roll_no, name = STUDENT_DATABASE[position_number]
            if position_number in present_students:
```

```
                lcd.clear()
                lcd.write_string(name[:16])
                lcd.cursor_pos = (1, 0)
                lcd.write_string("Already Marked!")
                print(f"[ALREADY MARKED] Roll: {roll_no} - {name}")
            else:
                present_students.add(position_number)
                lcd.clear()
                lcd.write_string(name[:16])
                lcd.cursor_pos = (1, 0)
                lcd.write_string("Status: Present")
                print(f"[PRESENT] Roll: {roll_no} - {name}")
        else:
            lcd.clear()
            lcd.write_string("Fingerprint")
            lcd.cursor_pos = (1, 0)
            lcd.write_string("Not Found / Unreg")
            print("[UNKNOWN] Fingerprint not registered!")

        time.sleep(3)

except KeyboardInterrupt:
    print("\\nSession Terminated. Generating Report...")

finally:
    lcd.clear()
    lcd.write_string("Generating Report")

    current_date = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # Absentees = all registered students - present students
    all_ids = set(STUDENT_DATABASE.keys())
    absent_ids = all_ids - present_students

    # Generate attendance report
    report = [
        "=== ATTENDANCE REPORT ===",
        f"Date/Time: {current_date}",
        f"Total Students : {len(STUDENT_DATABASE)}",
        f"Present : {len(present_students)}",
        f"Absent : {len(absent_ids)}",
        "",
        "--- PRESENT STUDENTS ---"
    ]

    for fingerprint_id in sorted(present_students):
        roll, name = STUDENT_DATABASE[fingerprint_id]
        report.append(f"- [{roll}] {name}")

    report.append("")
    report.append("--- ABSENTEE LIST ---")

    for fingerprint_id in sorted(absent_ids):
        roll, name = STUDENT_DATABASE[fingerprint_id]
        report.append(f"- [{roll}] {name}")

    report_text = "\\n".join(report)

    # Save report to a local text file
    with open("attendance.txt", "w", encoding="utf-8") as file:
        file.write(report_text)

    # Send report through Telegram
    try:
        bot.send_message(
            chat_id=TELEGRAM_CHAT_ID,
            text=report_text
        )

        lcd.clear()
        lcd.write_string("Report Sent!")
        lcd.cursor_pos = (1, 0)
        lcd.write_string("Saved to File")
        print("\\n&gt;&gt; Report Sent via Telegram &amp;amp; Saved to attendance.txt!")
    except Exception as e:
        lcd.clear()
        lcd.write_string("Telegram Error!")
        print(f"\\n&gt;&gt; Telegram Error: {e}")

    time.sleep(2)
    lcd.clear()
    GPIO.cleanup()
```